



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

LENGUAJES Y GRAMÁTICAS FORMALES

LOS ASTRONAUTAS

“EXPLORANDO EL UNIVERSO DE LOS LENGUAJES, UN NODO A LA VEZ”

PROFESOR:
FÉLIX MÁRQUEZ

PRESENTADO POR:

C.I.: V-25.933.680. ALBURQUERQUE SHEEN

C.I.: V-30.907.427. ANTOIMA MARIANGEL

C.I.: V-28.475.271. GARCÍA CARLOS

C.I.: V-24.848.424. VARGUILLAS GÉNESIS



Fundamentos y Jerarquía

(Teoría Aplicada)

RELACIÓN GRAMÁTICA -

GRAMÁTICA

$G = (N, \Sigma, P, S)$

- N = No terminales
- Σ = Terminales (alfabeto)
- P = Reglas de producción
- S = Símbolo inicial

Es el manual de instrucciones que define cómo formar cadenas.

LENGUAJE FORMAL

$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$

Es el conjunto MATEMÁTICO de todas las cadenas válidas que genera la gramática.

LA GRAMÁTICA GENERA EL LENGUAJE

EJEMPLO PRÁCTICO DE DERIVACIÓN

GRAMÁTICA:

$S \rightarrow NP VP$
 $NP \rightarrow \text{arroz} \mid \text{trigo}$
 $VP \rightarrow \text{crece}$

DERIVACIÓN DE "arroz crece":

$S \Rightarrow NP VP \Rightarrow \text{arroz} VP \Rightarrow \text{arroz crece}$

DERIVACIÓN DE "trigo crece":

$S \Rightarrow NP VP \Rightarrow \text{trigo} VP \Rightarrow \text{trigo crece}$

LENGUAJE GENERADO:

$L(G) = \{ \text{arroz crece}, \text{trigo crece} \}$

JERARQUÍA DE CHOMSKY : TIPO 3

TIPO 3: REGULAR

REGLAS: $A \rightarrow aB$ o $A \rightarrow a$

AUTÓMATA:

Autómata Finito

APLICACIÓN:

Análisis Léxico (tokens, palabras clave)

EJEMPLO BNF (Números enteros):

$\langle \text{dígito} \rangle ::= "0" \mid "1" \mid "2" \mid "3" \mid "4" \mid "5" \mid "6" \mid "7" \mid "8" \mid "9"$

$\langle \text{entero} \rangle ::= \langle \text{dígito} \rangle \mid \langle \text{dígito} \rangle \langle \text{entero} \rangle$

Genera: 0, 5, 123, 9999

JERARQUÍA DE CHOMSKY : TIPO 2

TIPO 2: LIBRE DE CONTEXTO

REGLAS:

$A \rightarrow \gamma$ (cualquier combinación de terminales y no terminales)

AUTÓMATA:

Autómata de Pila

APLICACIÓN:

Análisis Sintáctico (estructura del código)

EJEMPLO BNF (Paréntesis balanceados):

$\langle S \rangle ::= "(" \langle S \rangle ")" \langle S \rangle \mid \epsilon$

Genera: (), (()), ()(), ()()

NOTA: Un Autómata Finito (Tipo 3) NO puede generar paréntesis balanceados

JERARQUÍA DE CHOMSKY : TIPO 1

TIPO 1: SENSIBLE AL CONTEXTO

REGLAS:

$\alpha A \beta \rightarrow \alpha \gamma \beta$ (depende del contexto)

AUTÓMATA:

Autómata Linealmente Acotado (LBA)

EJEMPLO:

$a^n b^n c^n$ (igual número de a, b y c)

Kozen (1997):

Una gramática Tipo 2 NO puede generar este lenguaje

JERARQUÍA DE CHOMSKY : TIPO 0

TIPO 0: SIN RESTRICCIONES

REGLAS:

$\alpha \rightarrow \beta$ (sin limitaciones)

AUTÓMATA:

Máquina de Turing

EJEMPLO :

Cualquier lenguaje Turing-completo (C++, Python, Java, etc.)

Sipser (2012): Representa el máximo poder computacional

NOTA IMPORTANTE: Tipo 3 $\subset$ Tipo 2 $\subset$ Tipo 1 $\subset$ Tipo 0

Cada tipo es subconjunto del tipo superior

TABLA RESUMEN - JERARQUÍA DE CHOMSKY

Tipo	Nombre	Restricción de Producciones	Lenguaje Generado	Autómata Reconocedor
Tipo 3	Regular	$A \rightarrow aB$ o $A \rightarrow a$ (con $A, B \in N, a \in \Sigma$)	Lenguaje Regular	Autómata Finito
Tipo 2	Libre de Contexto (GLC)	$A \rightarrow \gamma$ ($A \in N, \gamma \in (N \cup \Sigma)^*$)	Lenguaje Libre de Contexto	Autómata de Pila (PDA)
Tipo 1	Sensible al Contexto (GSC)	$\alpha A \beta \rightarrow \alpha \gamma \beta$ con $\gamma \neq \epsilon$ (	$\alpha A \beta$	$\leq$
Tipo 0	Sin Restricciones	$\alpha \rightarrow \beta$ (α debe contener al menos un no terminal)	Lenguaje Recursivamente Enumerable	Máquina de Turing

A MAYOR POTENCIA → MAYOR COMPLEJIDAD COMPUTACIONAL

¿POR QUÉ ES IMPORTANTE?

1) CLASIFICA

ANÁLISIS LÉXICO → TIPO 3

ANÁLISIS SINTÁCTICO → TIPO 2

2) LIMITA

Expresiones Regulares NO puede validar paréntesis balanceados

3) PREVIENE

GRAMÁTICA mal diseñada puede causar:

- Ambigüedad
- Bucles infinitos
- Código válido rechazado

LA JERARQUÍA DE CHOMSKY ES UN PILAR FUNDAMENTAL DE LA TEORÍA DE LA COMPUTACIÓN Y LOS COMPILADORES.

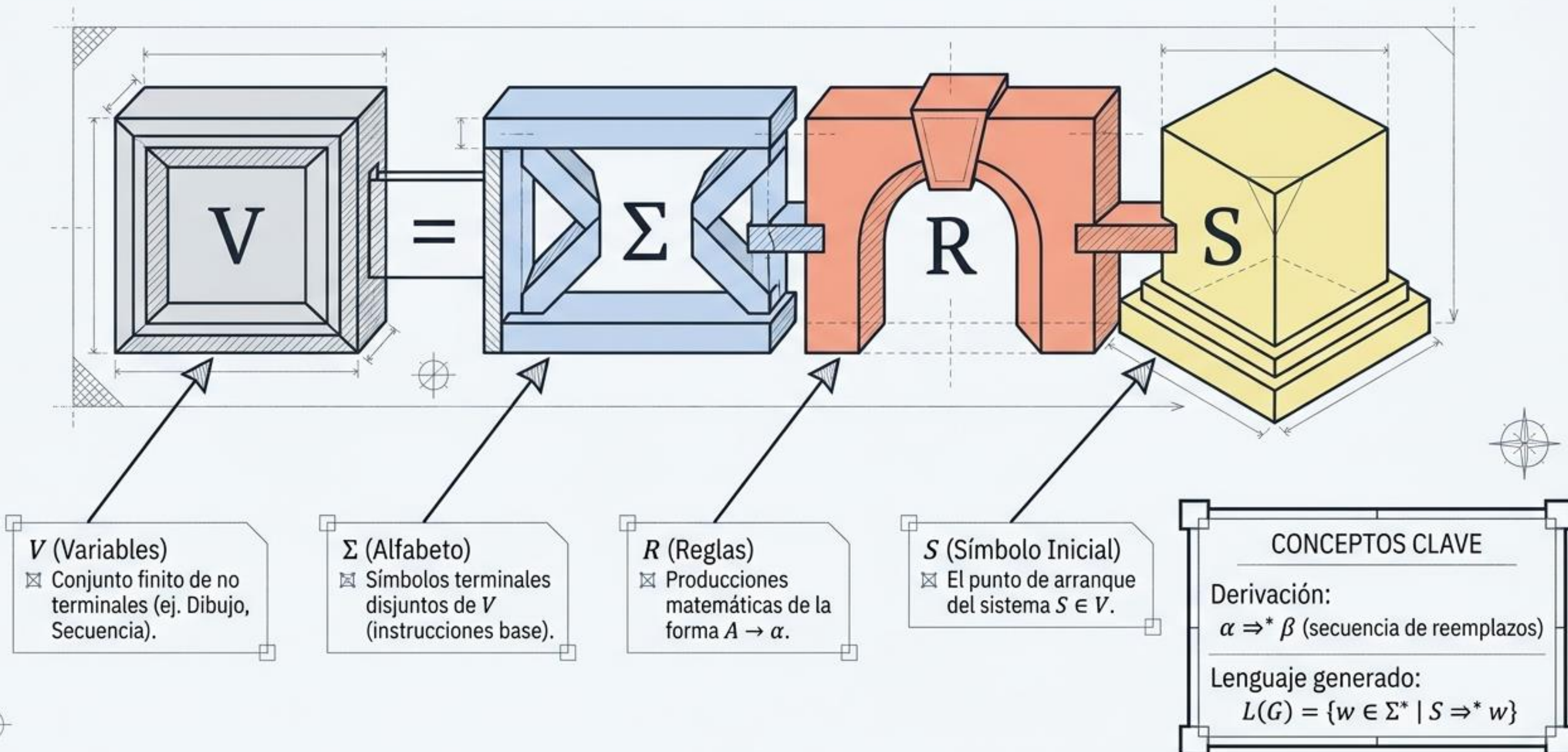
Derivación y Modelado

(Caso Práctico: Genoma)



Fundamentos Teóricos

La Base Matemática: Gramáticas Libres de Contexto (GLC)



El Alfabeto del Genoma

Traducción del alfabeto ADN {a, c, g, t} a comandos espaciales (Turtle Graphics)

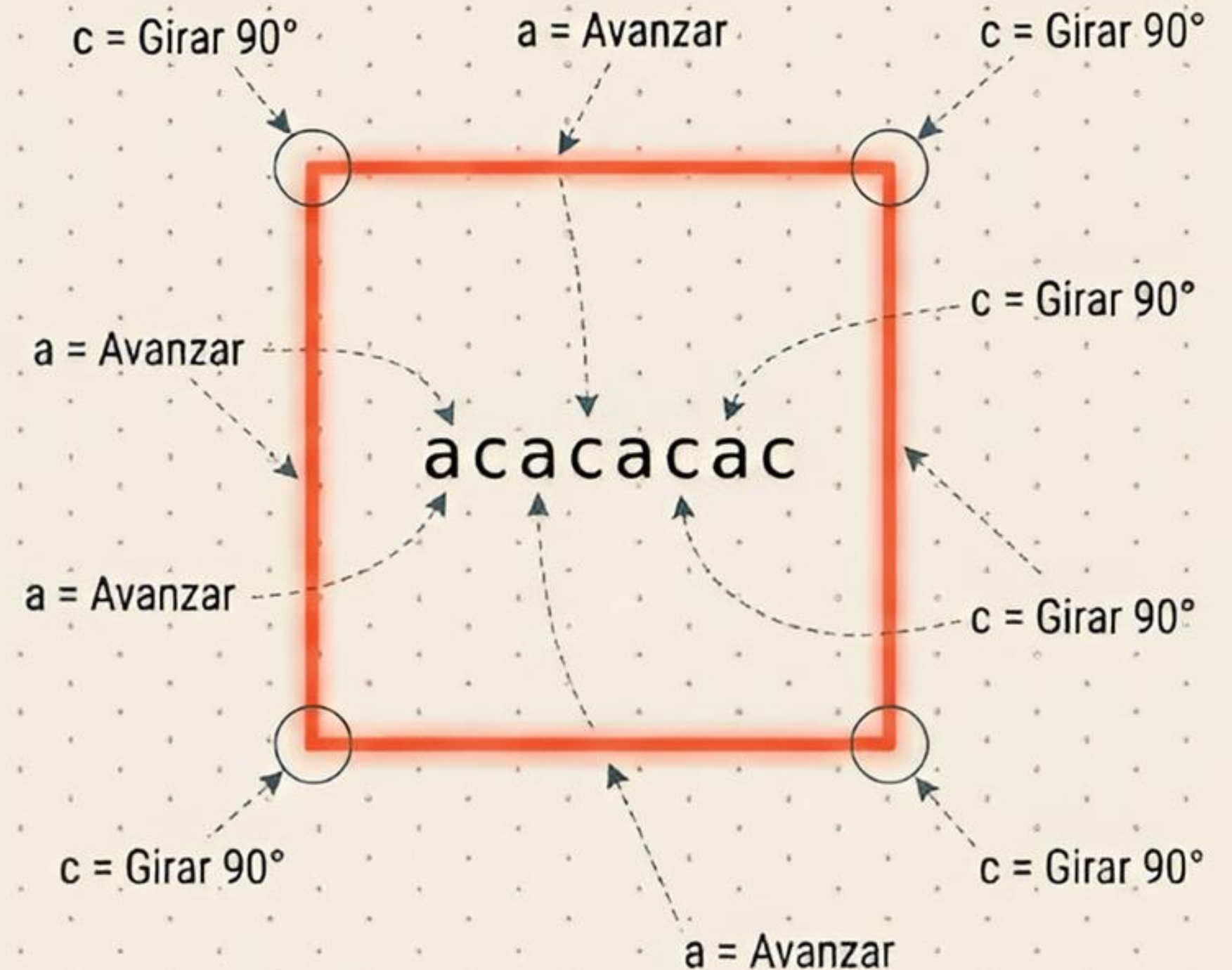
a	c	g	t
(Avanzar/Dibujar)	(Girar)	(Abrir Rama)	(Cerrar Rama)
Dibuja segmento unitario.	Rota 90° en sentido horario sin dibujar.	Guarda posición/orientación en la pila y desvía el trazo.	Recupera posición/orientación guardadas de la pila.
Equivalente Turtle: F	Equivalente Turtle: +	Equivalente Turtle: [	Equivalente Turtle:]
			

Regla de Interpretación: El primer símbolo “a” ejecutado inmediatamente después de “t” baja automáticamente el lápiz

Derivación I: Cuadrado

S $\Rightarrow$ (1) CUADRADO
 $\Rightarrow$ (2) LADO LADO LADO LADO
 $\Rightarrow$ (3x4) a c a c a c a c

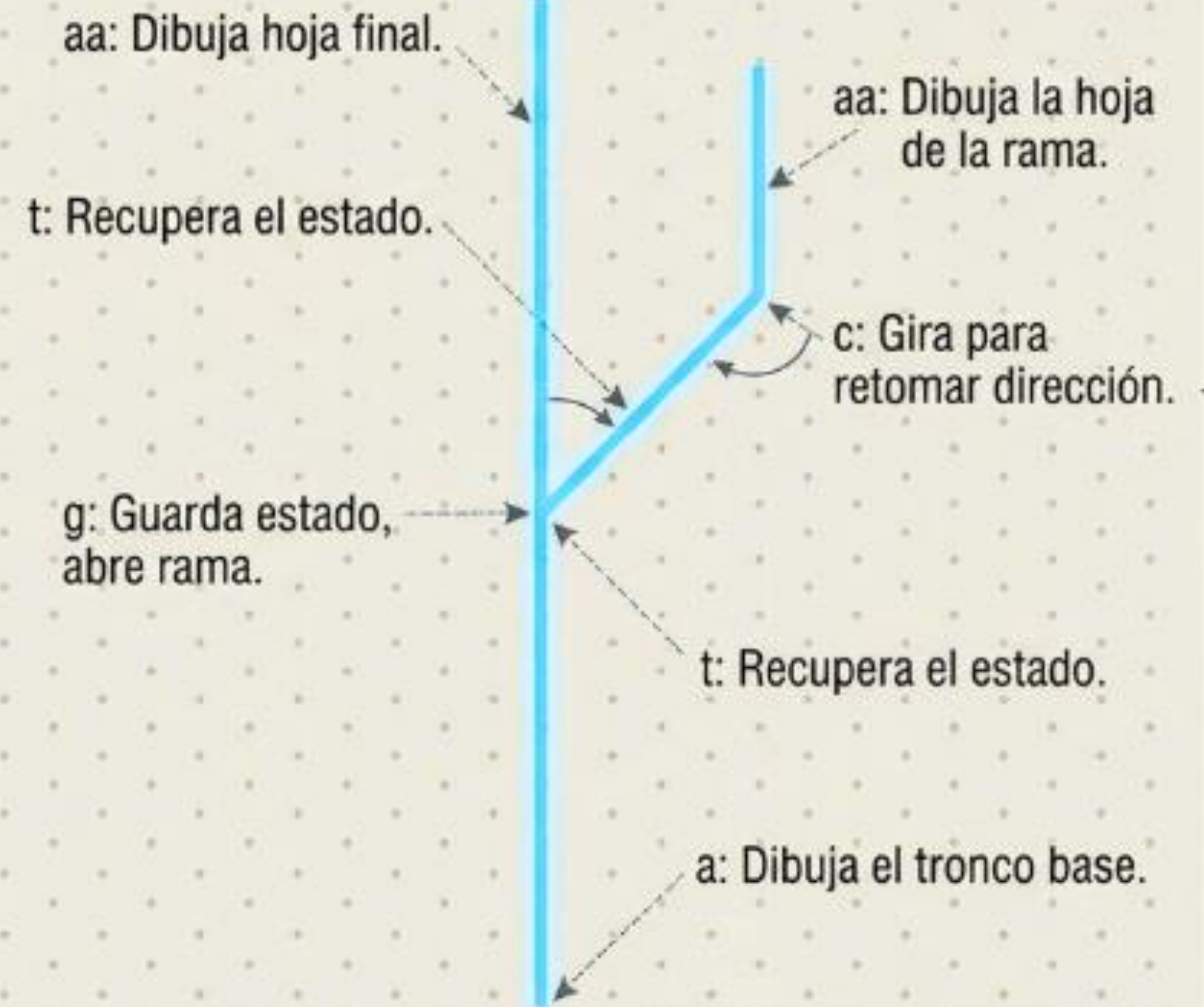
Cadena final: acacacac



Derivación II: Árbol Simple

S $\Rightarrow$ (4) ARBOL
 $\Rightarrow$ (5) a RAMA
 $\Rightarrow$ (7) a g ARBOL t c ARBOL
... (reemplazos iterativos)
 $\Rightarrow$ (9) a g a a t c a a

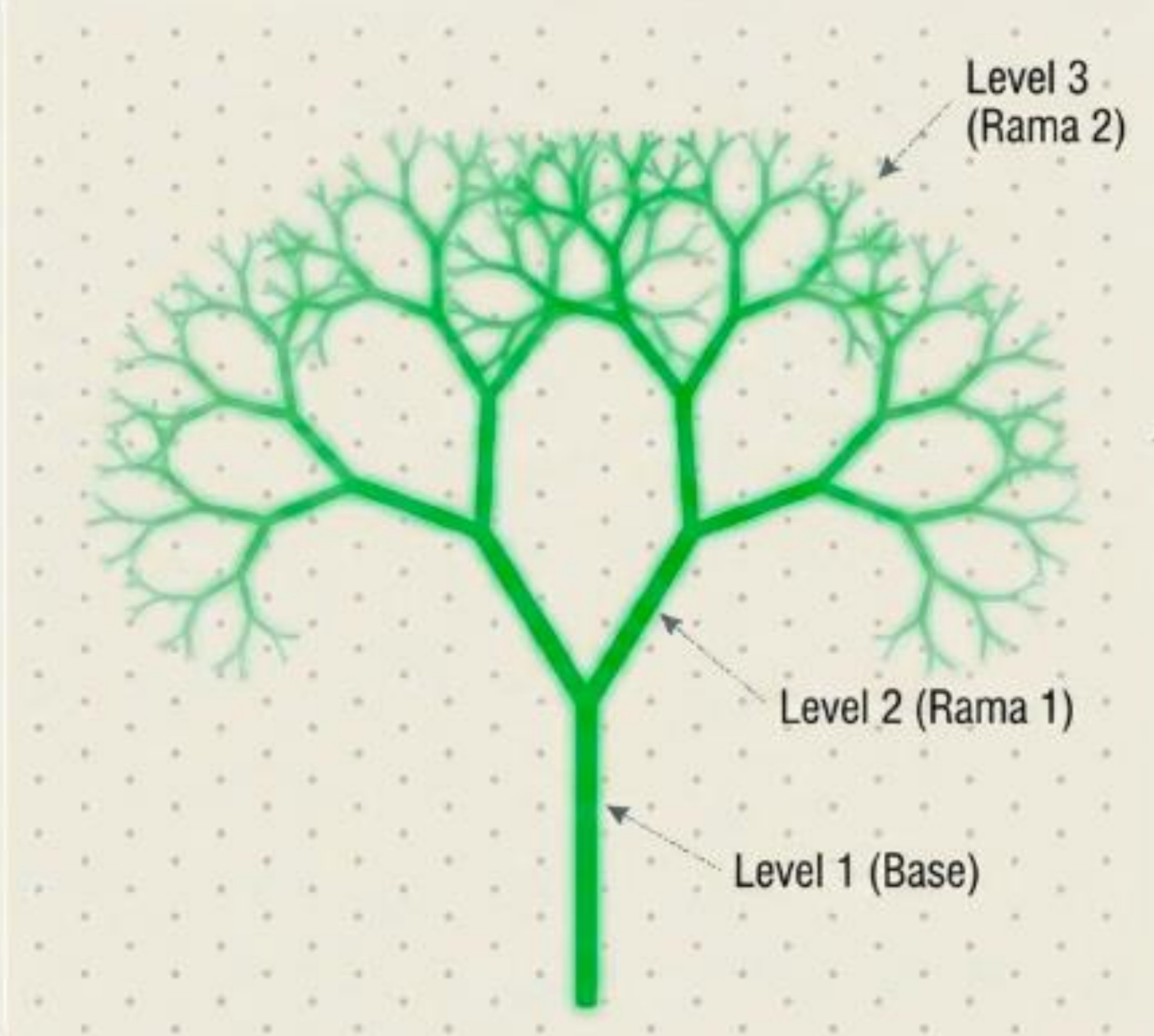
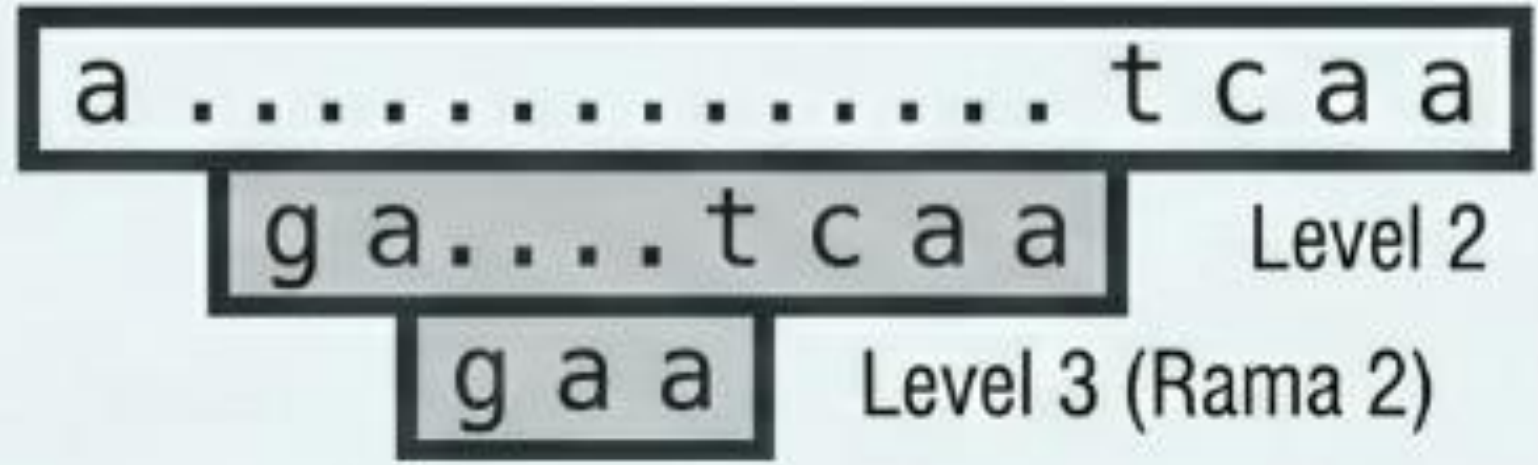
Cadena final: agaatcaa



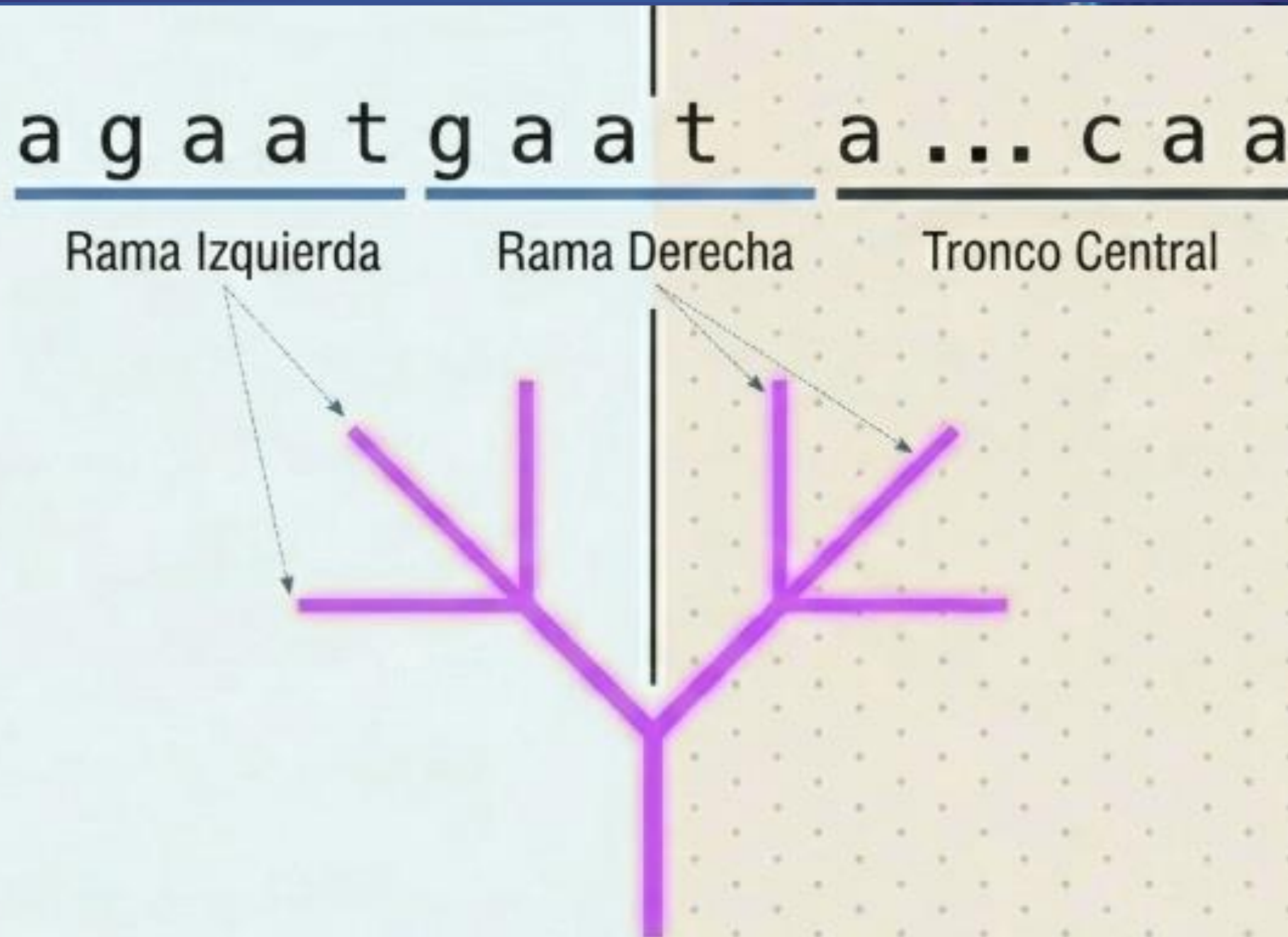
Derivación III: Árbol con Ramificación Anidada

agagaatcaatcaa

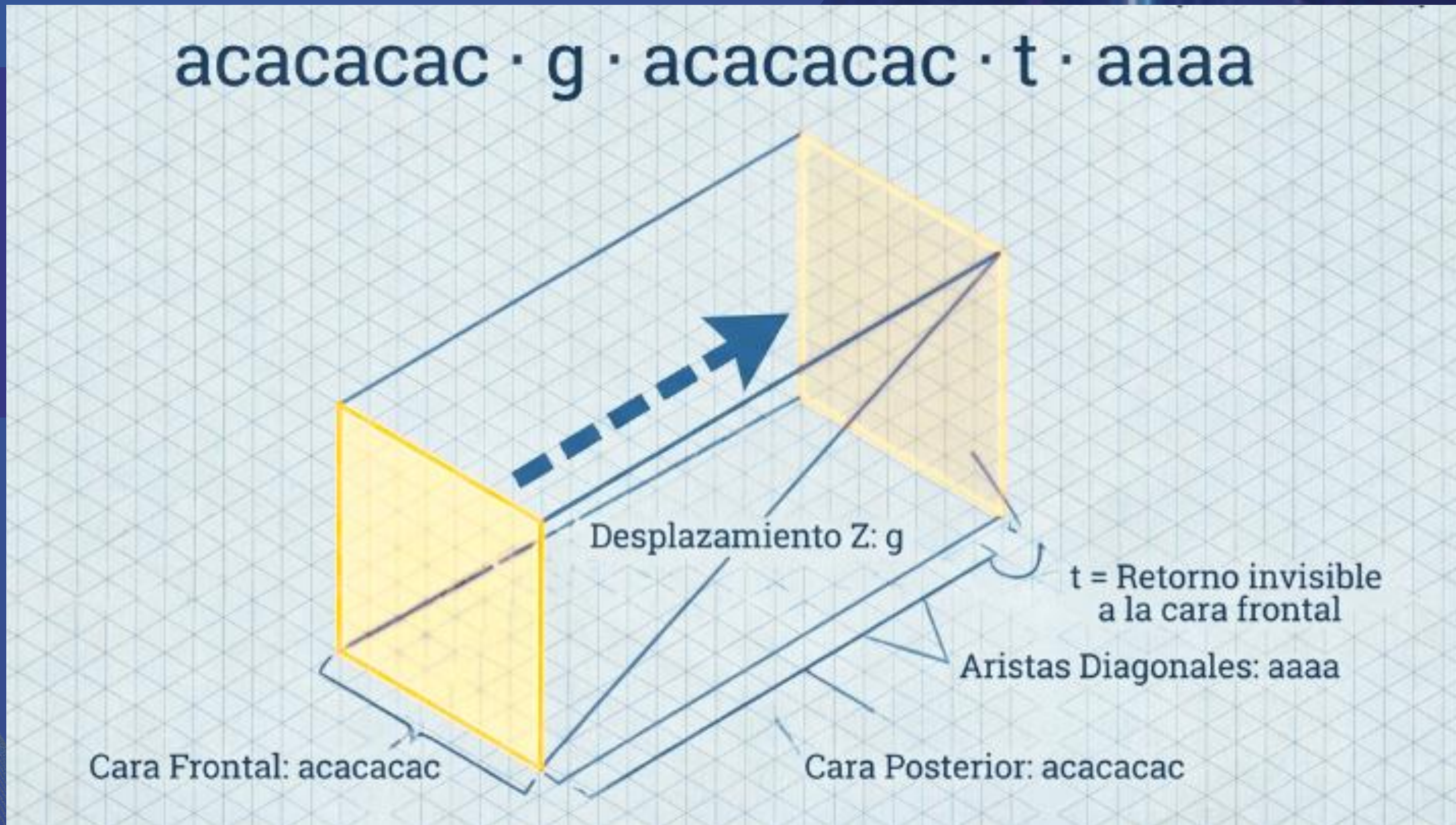
Depth-Chart



Derivación IV: Simetría Bilateral



Derivación V: Cubo (Proyección Isométrica)



Modelado: De la derivación a la representación gráfica



Limitaciones de la GLC

Limitación Estructural		Impacto en Modelado	Evolución / Solución
	Sin repetición numérica	No se puede expresar a^5 de forma compacta.	Añadir reglas con índices o gramáticas parametrizadas.
	Sin variables de estado	La posición y orientación espacial no existen dentro de la gramática.	Utilizar gramáticas dependientes del contexto o atributos.
	Sin control complejo	Falta de bucles condicionales; limitado a recursión lineal pura.	Extender a sistemas de reescritura de grafos (L-Systems avanzados).
	Ambigüedad Sintáctica	Múltiples derivaciones distintas pueden generar la misma figura.	Afecta al análisis del compilador, pero no interfiere con el renderizado visual.



Higiene y Optimización de Gramática

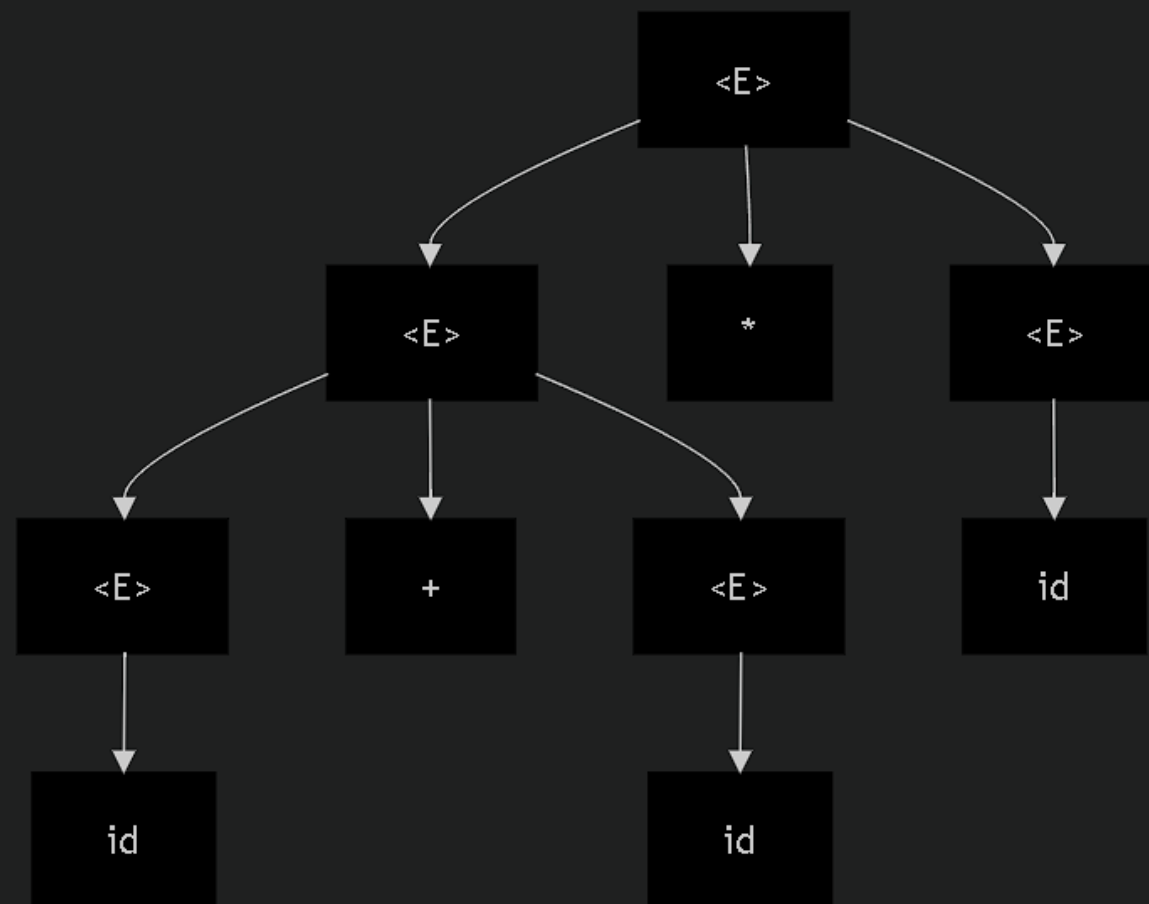
Gramática Ambigua

Ejemplo:

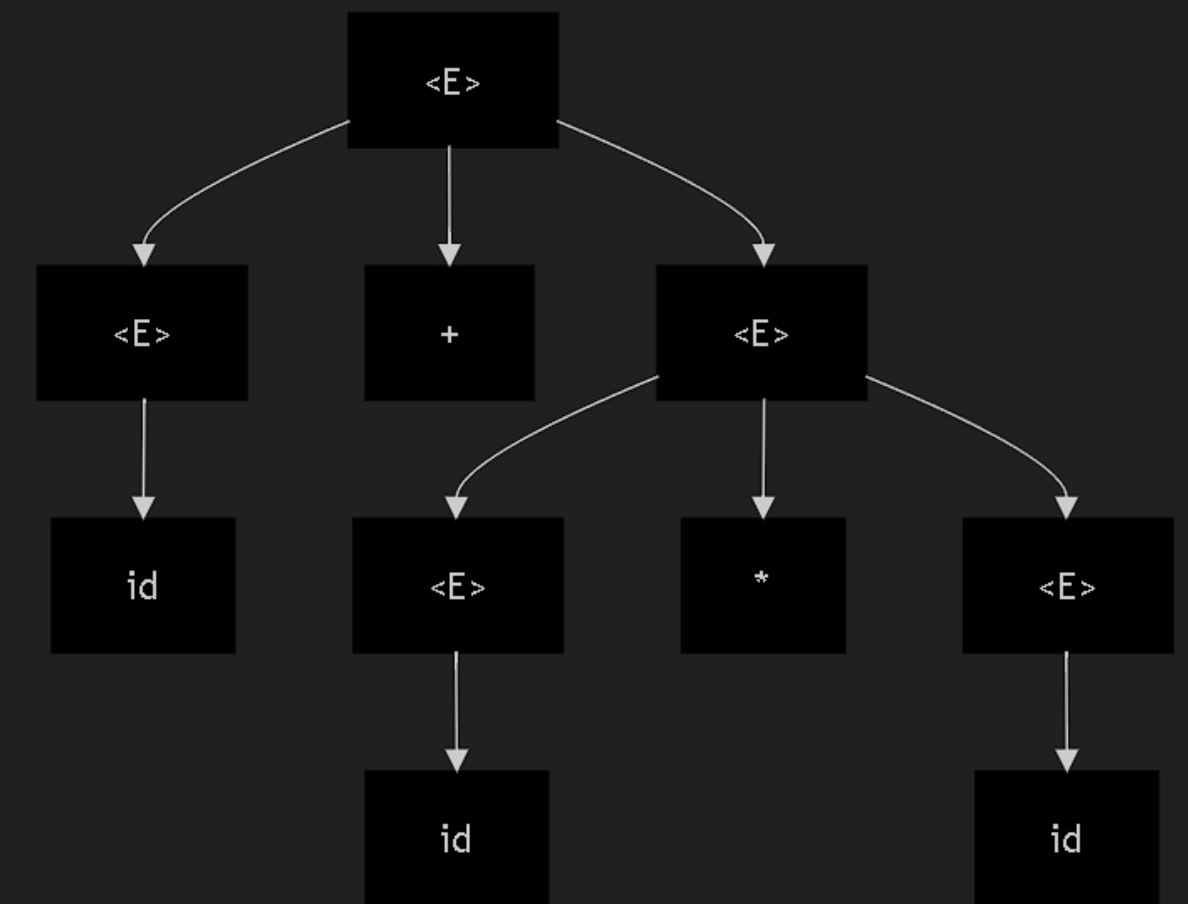
$\langle E \rangle ::= \langle E \rangle + \langle E \rangle \mid \langle E \rangle * \langle E \rangle \mid$

Cadena ambigua: $\text{id} + \text{id} * \text{id}$

Árbol de derivación 1 (interpretación: $(\text{id} + \text{id}) * \text{id}$)



Árbol de derivación 2 (interpretación: $\text{id} + (\text{id} * \text{id})$)



Solución:

$\langle E \rangle ::= \langle E \rangle + \langle T \rangle \mid$

$\langle T \rangle$

$\langle T \rangle ::= \langle T \rangle * \langle F \rangle \mid \langle F \rangle$

$\langle E \rangle ::= \text{id} \mid "(" \langle E \rangle ")"$

Recursividad por la izquierda

Ejemplo:

$\langle E \rangle ::= \langle E \rangle + \langle T \rangle \mid \langle T \rangle$

$\langle T \rangle ::= \text{id}$

Cadena problemática: id + id +
id

Regla de la forma $A \rightarrow A \alpha \mid \beta$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$

Paso 1: Identificar

Regla: $\langle E \rangle \rightarrow \langle E \rangle + \langle T \rangle \mid$
 $\langle T \rangle$

$A = \langle E \rangle, \alpha = + \langle T \rangle, \beta = \langle T \rangle$

Paso 2:

Transformar

$\langle E \rangle \rightarrow \langle T \rangle \langle E' \rangle$

$\langle E' \rangle \rightarrow + \langle T \rangle \langle E' \rangle \mid \varepsilon$

**Paso 3: Gramática
resultante**

$\langle E \rangle \rightarrow \langle T \rangle \langle E' \rangle$

$\langle E' \rangle \rightarrow + \langle T \rangle \langle E' \rangle \mid \varepsilon$

$\langle T \rangle \rightarrow \text{id}$

Factorización por la izquierda

Ejemplo:

$\langle A \rangle ::= \text{id} = \langle E \rangle \mid \text{id} (\langle E \rangle)$

Regla de la forma

$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_n:$

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

Paso 1: Identificar

Prefijo común: $\alpha = \text{id}$

Sufijos: $\beta_1 = = \langle E \rangle$, $\beta_2 = (\langle E \rangle)$

Paso 2: Factorizar

$\langle A \rangle \rightarrow \text{id} \langle A' \rangle$

$\langle A' \rangle \rightarrow = \langle E \rangle \mid (\langle E \rangle)$

Paso 3: Gramática

optimizada

$\langle A \rangle \rightarrow \text{id} \langle A' \rangle$

$\langle A' \rangle \rightarrow = \langle E \rangle \mid (\langle E \rangle)$

Conclusión del Higiene y Optimización de Gramática

PATOLOGÍA	RIESGO	SOLUCIÓN
Ambigüedad	Múltiples árboles de derivación	Niveles de precedencia
Recursividad por la izquierda	Bucle infinito	Transformar $A \rightarrow \beta A'$
Factorización por la izquierda	Indecisión al leer prefijos	Extraer prefijo común α
✅ Gramática limpia = Analizador sintáctico eficiente		

De la Expresión al Autómata

(Caso Práctico: Ajedrez)

OBJETIVO DEL PROYECTO

Reconocimiento Léxico

El propósito es diseñar una Expresión Regular y un AFD capaces de identificar patrones léxicos compatibles con un subconjunto de la notación PGN.

- ✓ No valida la legalidad física de la jugada.
- ✓ Determina la pertenencia al lenguaje formal.
- ✓ Sigue la Jerarquía de Chomsky (Tipo 3).



SUBCONJUNTO PGN SIMPLIFICADO

Componente	Descripción	Ejemplo
Piezas	K, Q, R, B, N (Peón implícito)	N, Q
Coordenadas	Columnas (a-h) y Filas (1-8)	e4, b7
Acciones	Captura (x), Enroque corto (O-O)	exd5, O-O
Modificador	Jaque (+) al final del movimiento	Qh5+

CADENAS DE PRUEBA



Aceptadas

e4, d5, a3 (Peones)
Nf3, Bc4, Re1 (Piezas)
exd5, Qxe5 (Capturas)
O-O, Qh5+ (Enroque/Jaque)



Rechazadas

z9 (Fuera del tablero)
Pe4 (Notación incorrecta)
Nxxe5 (Doble captura)
O--O (Sintaxis inválida)

ALFABETO FORMAL Σ



Piezas

$\{K, Q, R, B, N\}$



Tablero

$\{a-h\} \cup \{1-8\}$



Símbolos

$\{x, +, O, -\}$

$\Sigma = \{K, Q, R, B, N, a, b, c, d, e, f, g, h, 1, 2, 3, 4, 5, 6, 7, 8, x, O, -, +\}$

DEFINICIÓN DE LA GRAMÁTICA

$$G = (V , \Sigma , P , S)$$

No Terminales (V)

- ✓ S: Símbolo inicial
- ✓ MOV: Derivación de jugada
- ✓ PEON / PIEZA: Tipos de movimiento
- ✓ COL / FILA: Terminales de posición

Reglas de Producción (P)

$S \rightarrow \text{MOV} \mid \text{MOV JAQUE}$

$\text{PEON} \rightarrow \text{COL FILA} \mid \text{COL x COL FILA}$

$\text{ENROQUE} \rightarrow \text{O-O}$

DISEÑO DE LA EXPRESIÓN REGULAR

Construcción del Patrón

Se agrupan los casos de peones, piezas y enroque en un grupo de captura, seguido por un símbolo de jaque opcional.

```
([a-h][1-8]|[a-h]x[a-h][1-8]|  
[KQRBN][a-h][1-8]|[KQRBN]x[a-h][1-  
8]|0-0)\+?
```


DISEÑO DE LA EXPRESIÓN REGULAR

Construcción del Patrón

Se agrupan los casos de peones, piezas y enroque en un grupo de captura, seguido por un símbolo de jaque opcional.

```
([a-h][1-8]|[a-h]x[a-h][1-8]|  
[KQRBN][a-h][1-8]|[KQRBN]x[a-h][1-  
8]|0-0)\+?
```


AUTÓMATA FINITO

Lógica del Reconocedor

El AFD procesa el flujo de entrada a través de estados determinísticos:

- ✓ **q0**: Estado inicial.
- ✓ **q1, q3, q6**: Rama de peones y capturas.
- ✓ **q2, q4, q5, q7**: Rama de piezas.
- ✓ **q8, q9**: Rama de enroque.
- ✓ **qF**: Estado de aceptación.

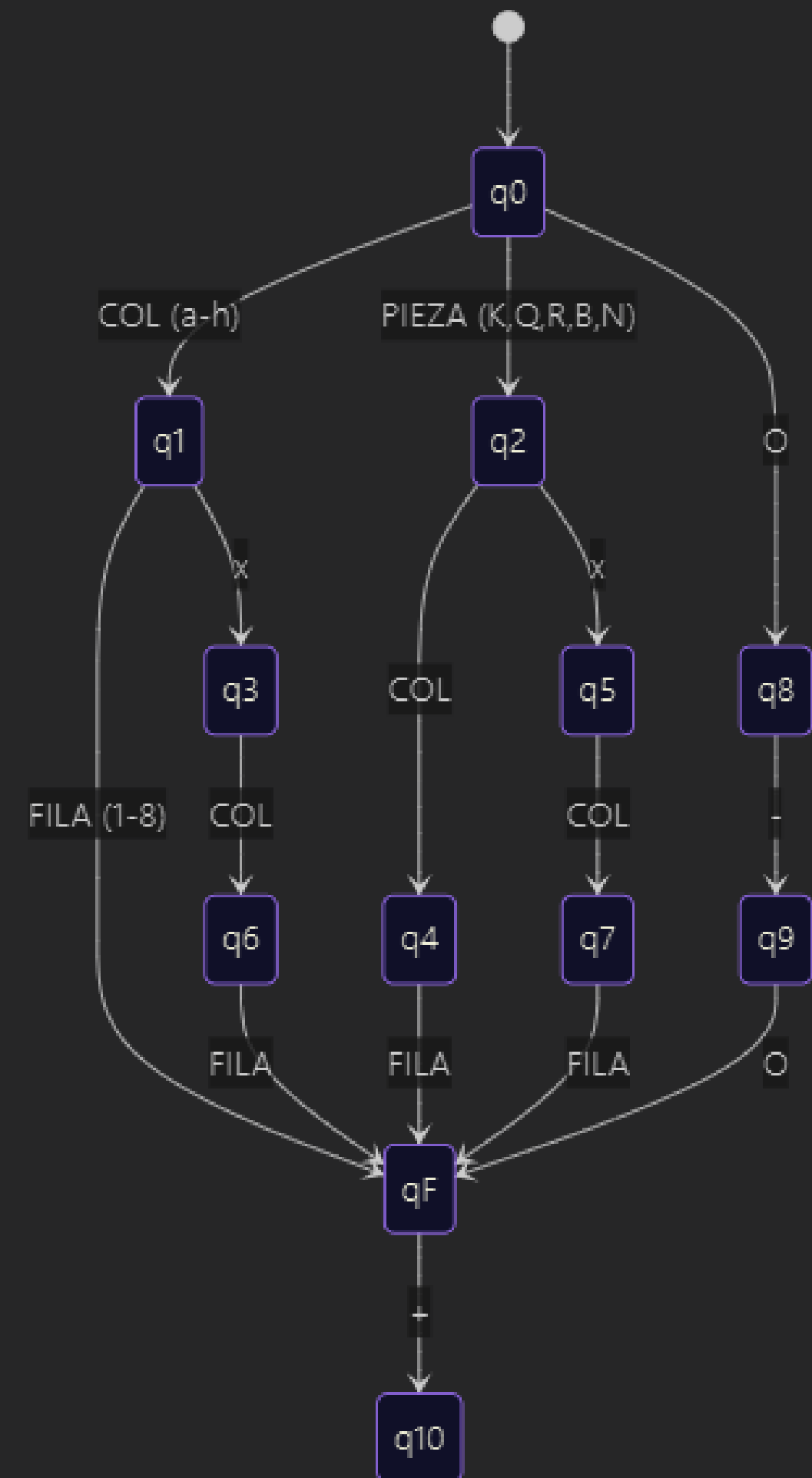


TABLA DE TRANSICIÓN FORMAL

Estado Actual	Entrada	Siguiente Estado
q0 (Inicio)	COL (a-h) / PIEZA / O	q1 / q2 / q8
q1 (Peón)	FILA (1-8) / x	qF (Acepta) / q3
q2 (Pieza)	COL / x	q4 / q5
q8 (Enroque)	-	q9
qF (Final)	+	q10 (Acepta)

The background is a dark, abstract composition. On the right side, a hand is shown reaching out, with fingers touching a complex, glowing network of lines and nodes. Various icons are scattered throughout the scene, including a speech bubble with a bar chart, a document with a checkmark, and a small robot-like figure. The overall color palette is dominated by deep blues, purples, and magentas, with bright highlights from the digital elements. The text is centered in a large, white, sans-serif font.

**¡Gracias por su
atención!**